ToiletTracker

1. Proposed Architecture

Architecture pattern: Layered client-server architecture

Status: **Planned.** Specific frameworks, database products, and map providers have not yet been selected.

Purpose and first-version scope

ToiletTracker is a mobile app for finding bathrooms on the Cal Poly SLO campus. Its first version will provide a campus map and bathroom search, community cleanliness and busyness ratings, and directions to a selected bathroom. All features will be available without creating an account. The team will enter the initial list of campus bathrooms and their details.

Major layers and components

Mobile client (presentation layer). Displays the map, bathroom search results, bathroom details, and ratings. It accepts reviews and requests directions. It uses the phone location service with permission and keeps an anonymous identifier for the app installation.

Backend API and service layer (application layer). The API receives requests from the app and returns results. Services handle bathroom searches, validate reviews, calculate rating summaries, and enforce the daily review limit. The service layer is an internal part of one backend application.

Database (data layer). Stores the team-entered bathroom locations and details, submitted cleanliness and busyness ratings, anonymous installation identifiers, and server-recorded review dates. Only the backend accesses the database.

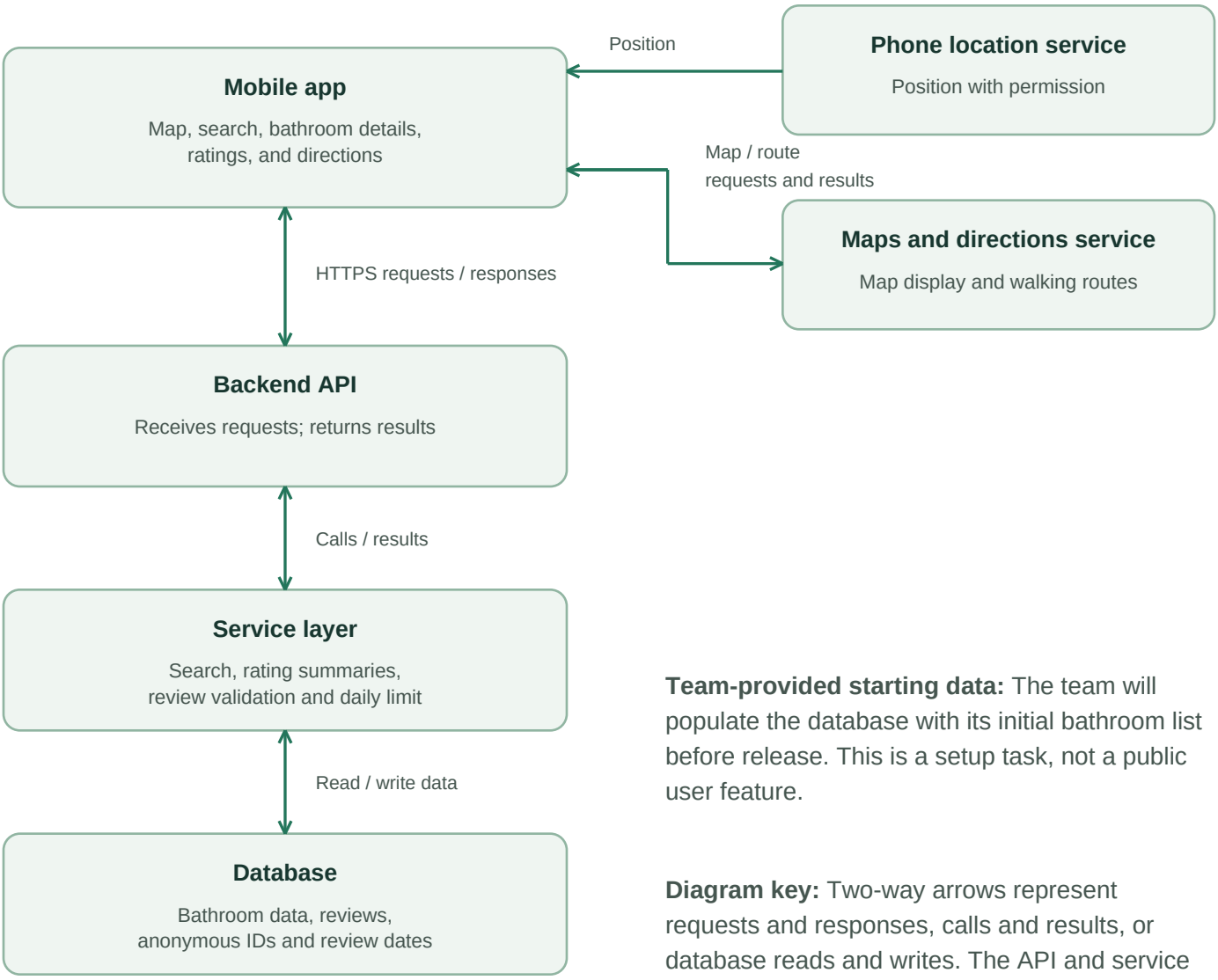
External services. A maps and directions service supplies the campus map and walking routes. The phone location service supplies the current position after permission is granted; this position does not need to be stored in the database.

Why this architecture fits

A shared backend and database let users see the same bathroom information and community ratings. Separating the interface, application logic, and stored data makes the app easier to maintain and lets team members work on different parts. Centralizing review checks in the backend applies the same rule to every submission. A single backend keeps the proposed design manageable for the team.

ToiletTracker

Architecture diagram and communication



Team-provided starting data: The team will populate the database with its initial bathroom list before release. This is a setup task, not a public user feature.

Diagram key: Two-way arrows represent requests and responses, calls and results, or database reads and writes. The API and service layer belong to the same backend.

Key communication flows

Find a bathroom: Mobile app → backend API → service layer → database; results return through the same layers. **Get directions:** The app sends the current position and bathroom destination to the maps service and displays its route.

Planned daily review rule

Allow one review containing cleanliness and busyness ratings per bathroom, per anonymous app installation, per campus calendar day (America/Los_Angeles). The backend uses server time and a database uniqueness rule to enforce the limit, including simultaneous submissions. This reduces repeat submissions; it cannot verify one person per device or guarantee truthful reviews. Reinstalling or using another device may bypass the limit.